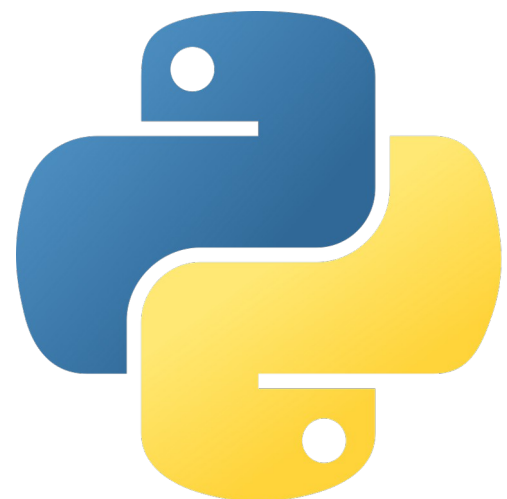


„Neuronale Nüsse“



PyTorch



TensorFlow

Projektziel

Einen Objekt klassifizierenden Lernalgorithmus zu entwickeln der durch überwachtes lernen die gängigsten Nussorten, zu erkennen lernt.

Training:

Datenvorbereitung -> (Trainieren -> Validieren)^x -> Speichern

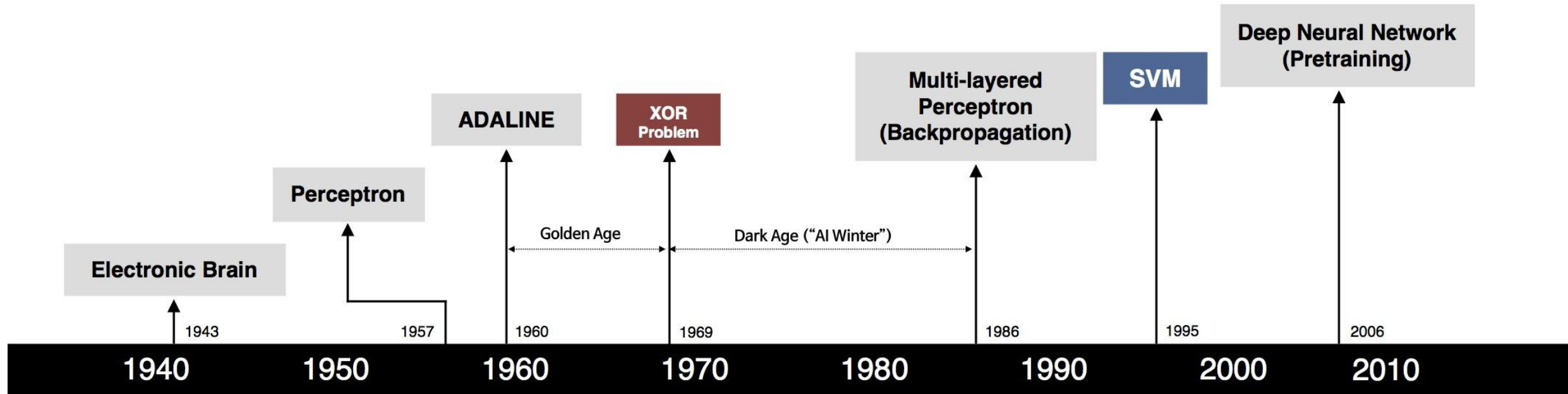
Detektion:

Videoeingabe -> Framevorbereitung -> Frameanalyse -> Aussage

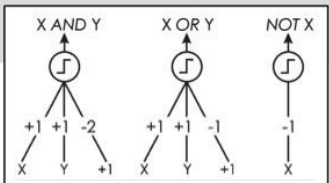
Inhaltsangabe

- Die Geschichte der KNN's
- Moderne Anwendungsgebiete
- Netztopologien/Netzarten/Beispiel Netzaufbau
- Propagation/Rückwärtspropagation/Formelsammlung
 - Einfaches Perzeptron/Adaline/Gefaltete Netze
 - Digitale Bilder
 - Verwendete Bibliotheken
 - Code
 - Aufgetretene Probleme
 - Quellen

Die Geschichte der KNN's



S. McCulloch – W. Pitts



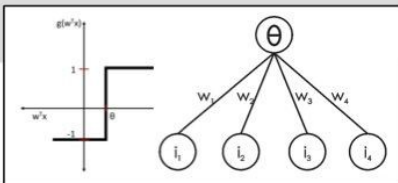
- Adjustable Weights
- Weights are not Learned



F. Rosenblatt



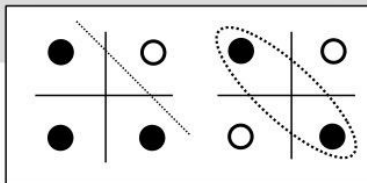
B. Widrow – M. Hoff



- Learnable Weights and Threshold



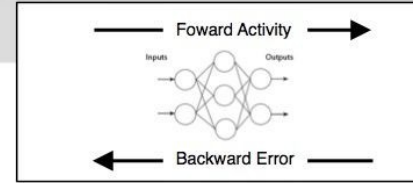
M. Minsky – S. Papert



- XOR Problem



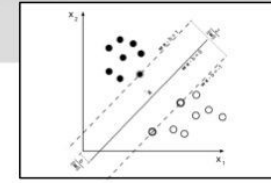
D. Rumelhart – G. Hinton – R. Williams



- Solution to nonlinearly separable problems
- Big computation, local optima and overfitting



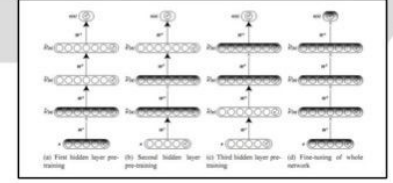
V. Vapnik – C. Cortes



- Limitations of learning prior knowledge
- Kernel function: Human Intervention



G. Hinton – S. Ruslan



- Hierarchical feature Learning

Warum Neuronal?

Neuronale Netze (IT) = Neuronales Netz (Biologie)

Die Neuronalen Netze sind von der Funktionsweise und Aufbau des Neuronalen Netzes im Gehirn, sehr dem Modell der Biologie nach empfunden.

Anders als „einfache“ Funktionen können KNN's nicht nur zwischen 0 und 1 unterscheiden sondern durch einstellbare Gewichte, anhand verschiedener Eigenschaften in Abstufungen entscheiden.

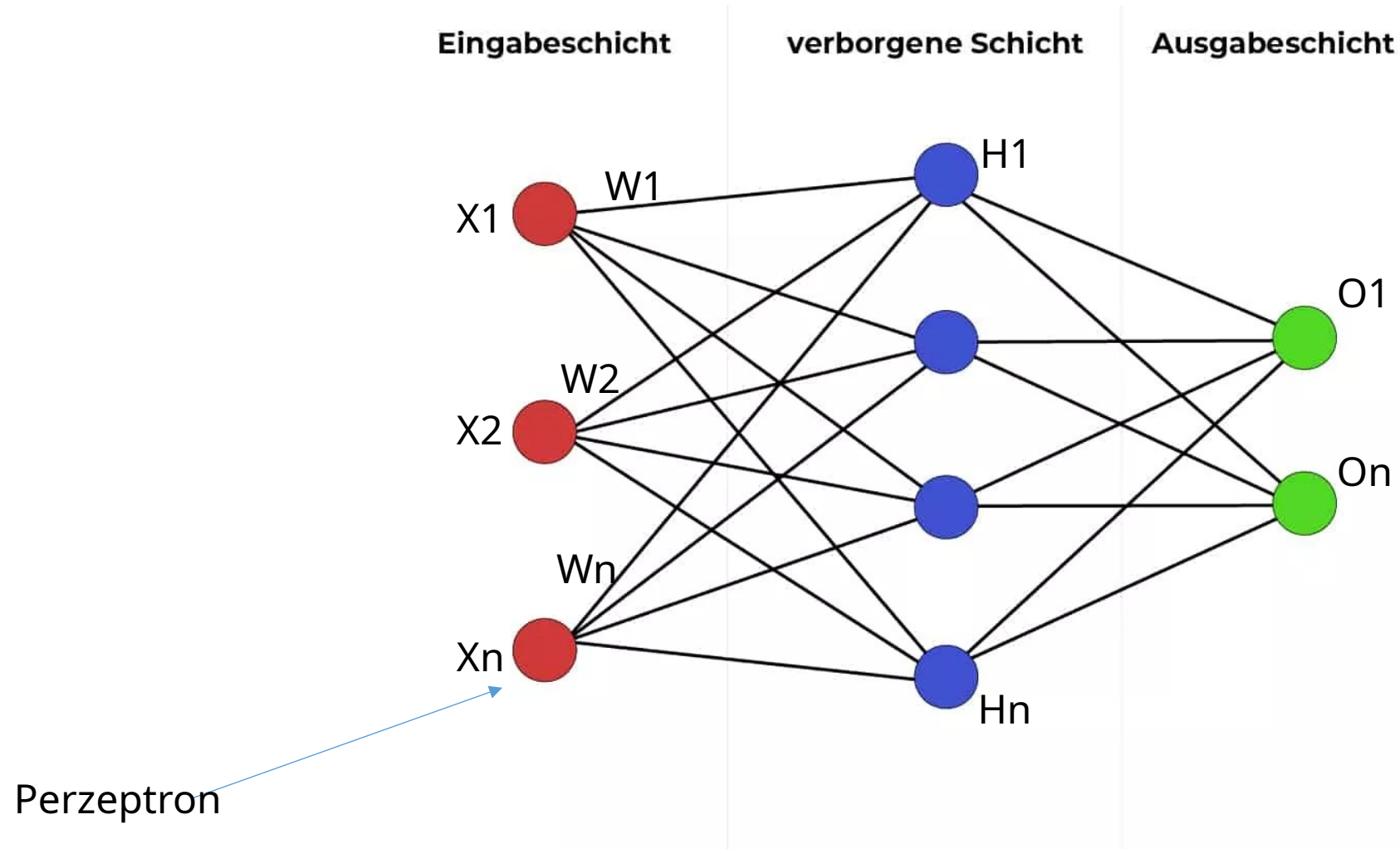
Moderne Anwendungsgebiete

- Fehlererkennung
 - Klassifikationen
- Bild/Sprachsynthese
 - Robotik
- Steuer und Regelungstechnik
 - Autonomes Fahren
- Medizinische Diagnostik

Bekannte Netzarten

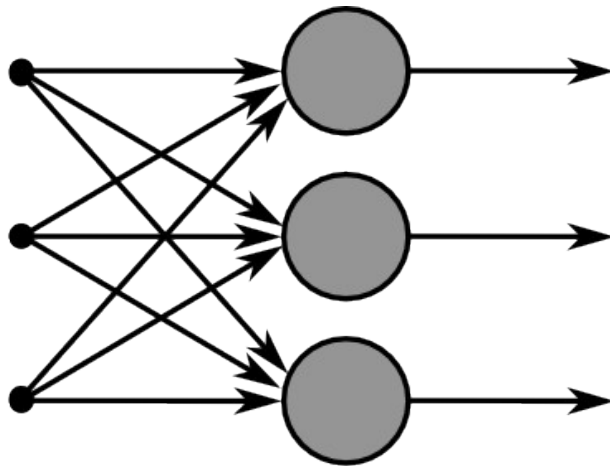
- Neocognitron(Mustererkennung)
- Hopfield Netze(Synchron/Asynchron)
- Faltende Neuronale Netze(Bildverarbeitung)
- Kohonen Netz (Selbstorganisierende Karten)

Beispiel Netzaufbau



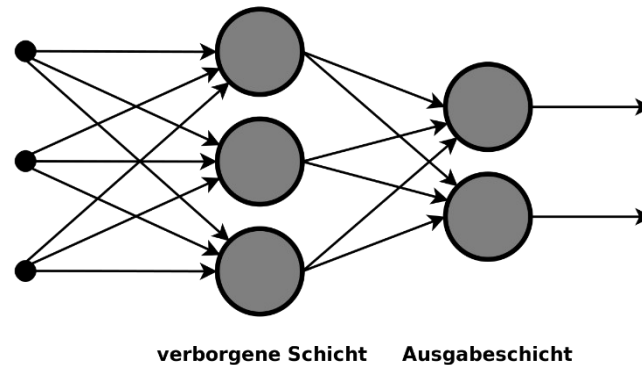
Netztopologien

Einschichtiges Netz

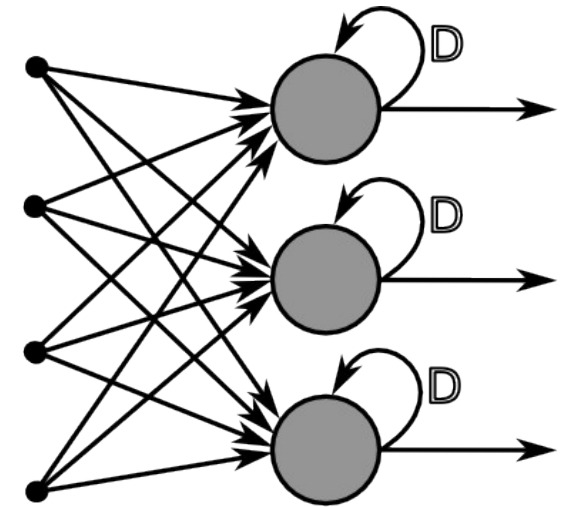


Ausgabeschicht

Mehrschichtiges Netz



Rekurrentes Netz



Perzeptron

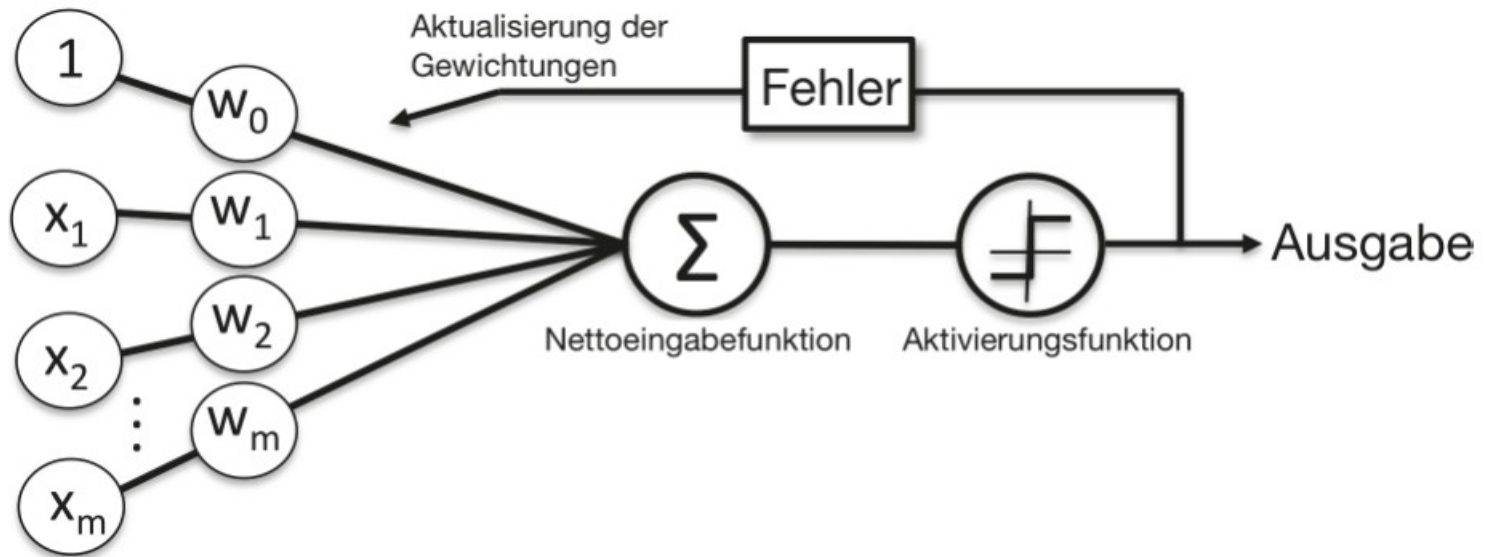
1) X_1 bis X_n = Arrays der Bilder
 $X_1 + X_2 + X_3 + \dots X_n$

2) W_1 bis W_n = Selbstjustierend.
Gewicht
Verrechnung mit den Inputs

3) Übertragungsfunktion = Linearkomb.
Summe = $(X_1 * W_1) + (X_2 * W_2) \dots + (X_n * W_n)$

4) Netzeingabe net_j = Gewichtete
Summe
 $X_n * W_n$

5) Aktivierungsfunktion = ReLu
(Rectifier) und Sigmoidsche ...



Formelsammlung

$$1) \quad \vec{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix},$$

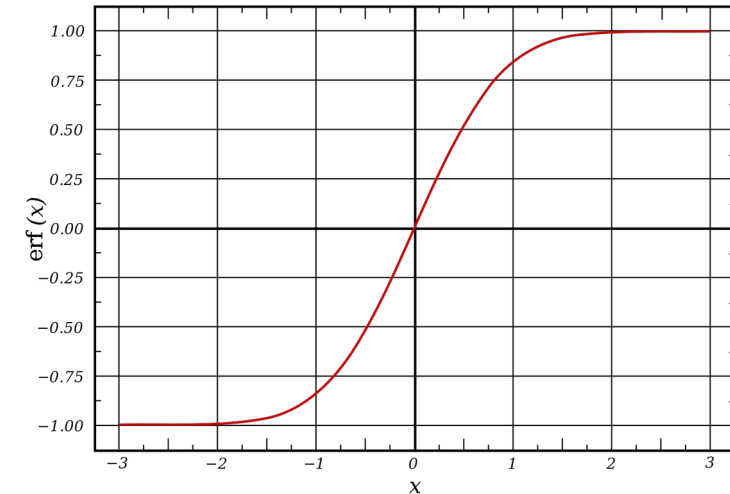
$$2) \quad \vec{x} \cdot \vec{y} = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \cdot \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} := x_1 y_1 + \dots + x_n y_n.$$

$$3) \quad y = \begin{cases} 1 & \text{if } u \geq \theta \\ 0 & \text{if } u < \theta \end{cases}$$

$$4) \quad u = \sum_{i=1}^n w_i x_i$$

$$5) \quad f'(x) = \begin{cases} 1 & \text{if } x > 0, \\ 0 & \text{if } x < 0. \end{cases} \quad \text{oder} \quad \text{sig}'(t) = \text{sig}(t) (1 - \text{sig}(t)).$$

Sigmoidsche Aktivierungsfunktion



Vorwärtspropagation

- Das Perzeptron verrechnet die Eingaben mit den Gewichten.
Daraus resultiert dann die Gewichtete Summe.
- Diese werden dann durch die
Aktivierungsfunktion weiter geschickt.
- Wenn der Wert ≥ 0 sendet das Perzeptron
eine 1 ansonsten eine 0

Rückwärtspropagation

- Das Perzeptron vergleicht erwarteten Output und gegebenen Output
- Mit ermitteltem Gradienten und gewähltem Optimierungsverfahren werden die Gewichte angepasst

$$E = \frac{1}{2} \sum_{i=1}^n (t_i - o_i)^2.$$

Dabei ist

E der Fehler,

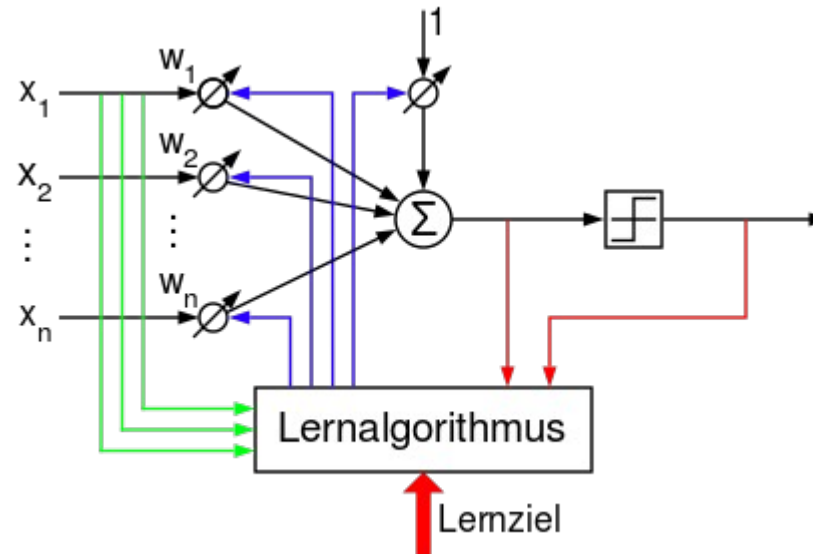
n die Anzahl der Muster, die dem Netz vorgestellt werden,

t_i die gewünschte Soll-Ausgabe oder Zielwert (*target*) und

o_i die errechnete Ist-Ausgabe (*output*).

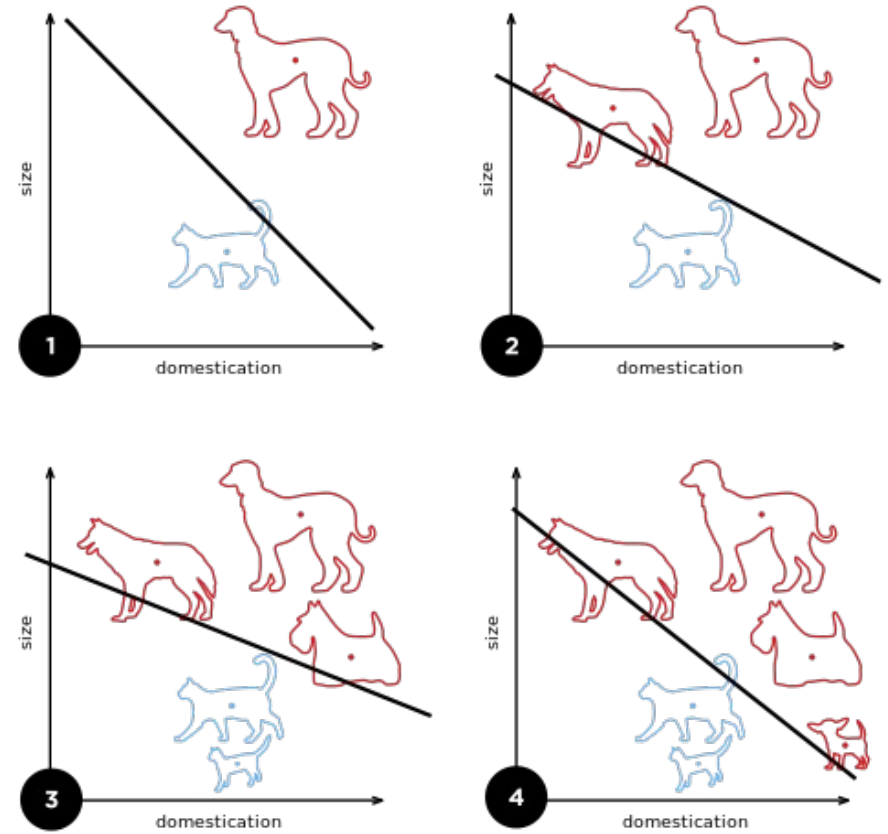
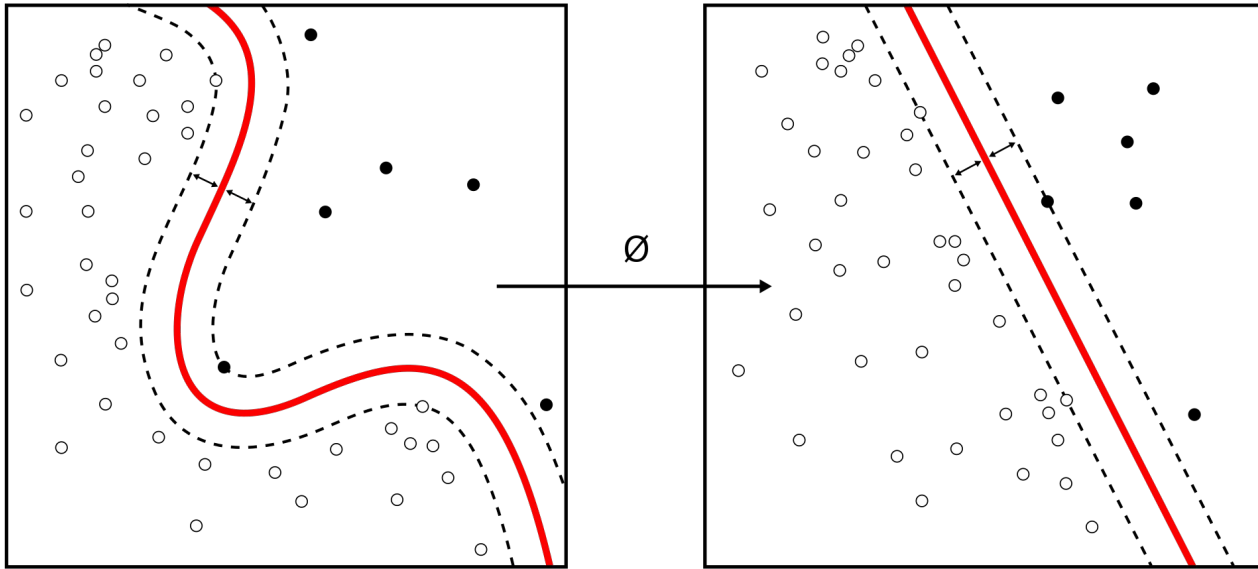
Adaline

- Adaptive linear Neuron = Adaptives lineares Neuron
- Aktivierung einer Ausgabeeinheit als Information, Fehlerrückführung

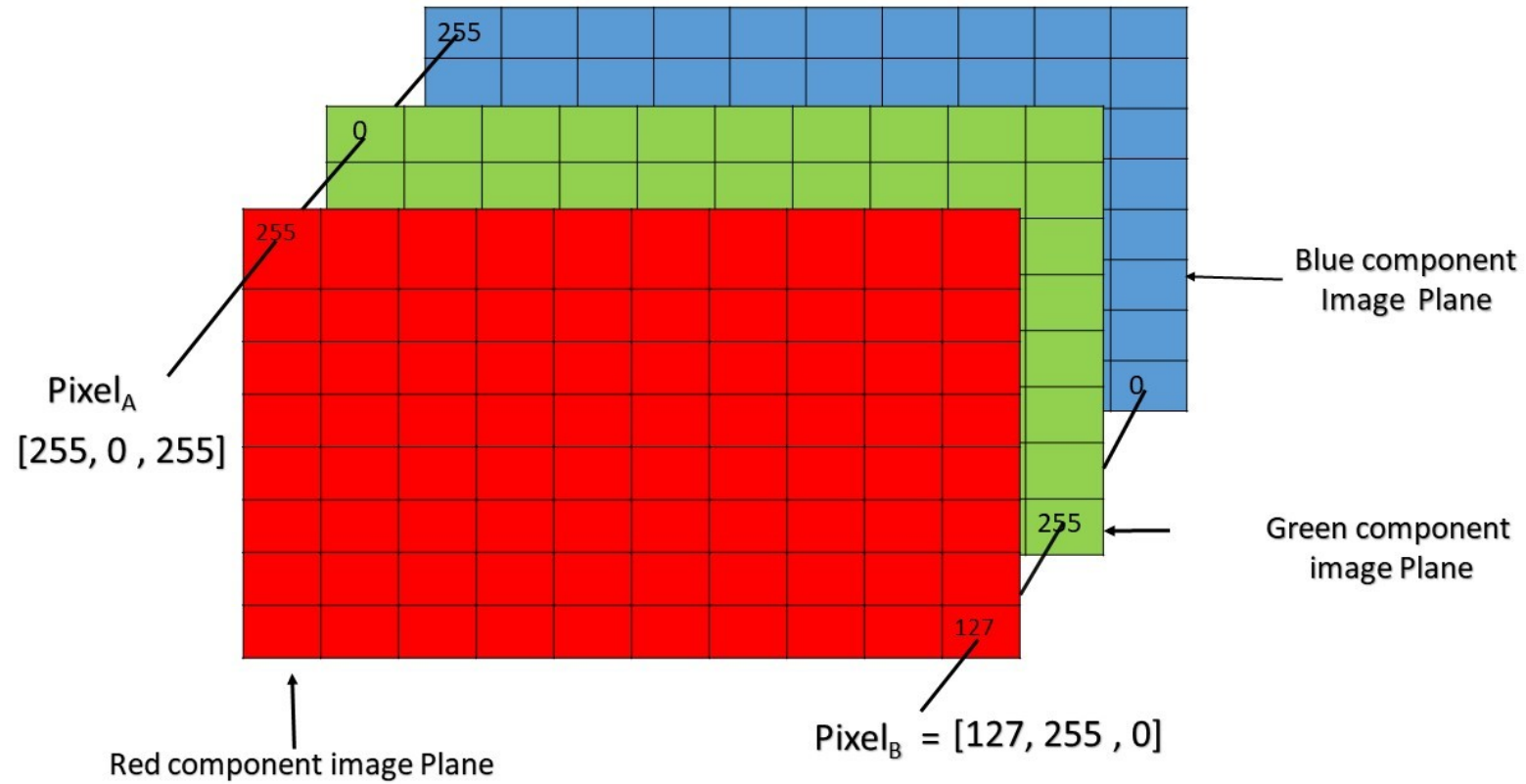


Eigenschaften

Scatter Plots

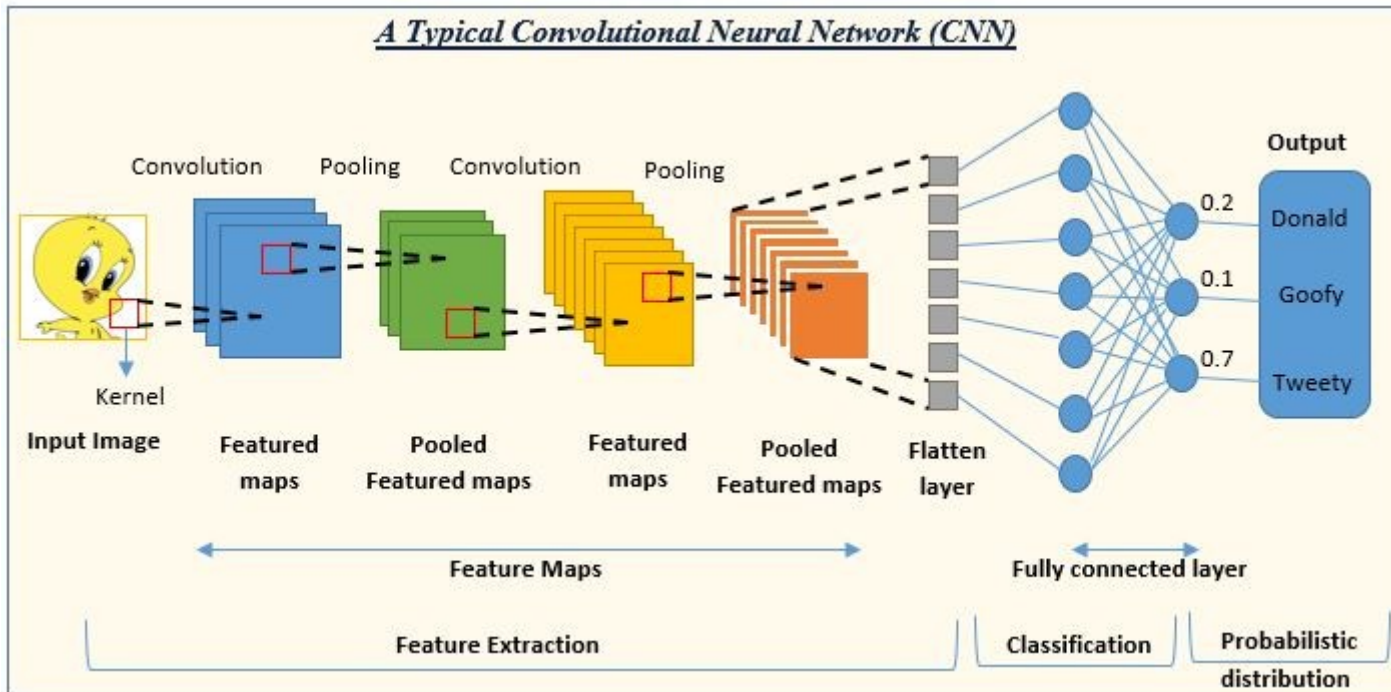


Digitale Bilder



Pixel of an RGB image are formed from the corresponding pixel of the three component images

Faltendes Netz



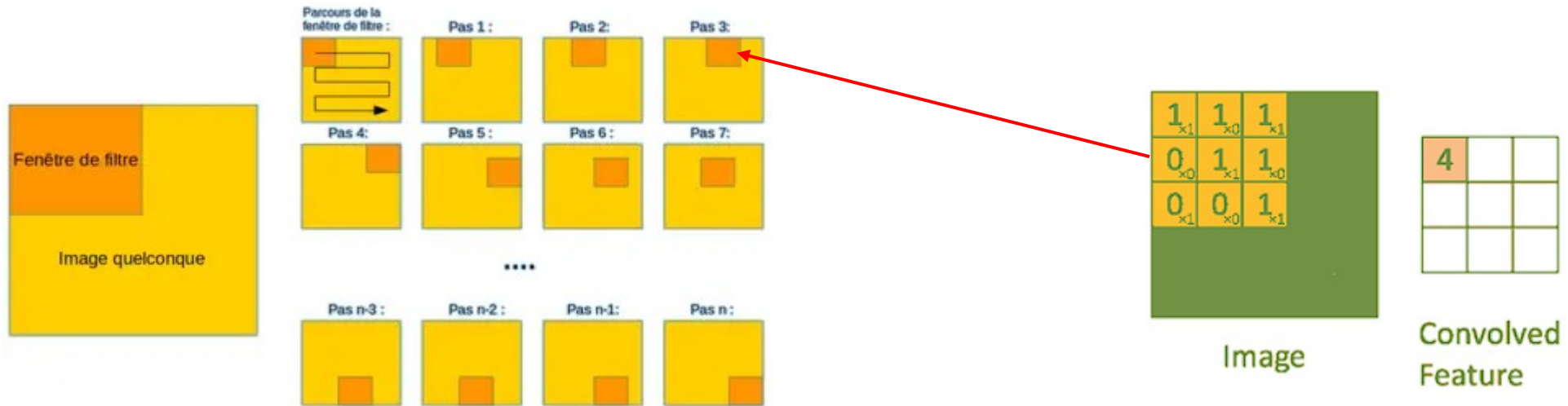
-Schichten zur Merkmalsextrahierung

-Schichten zu Klassifikation

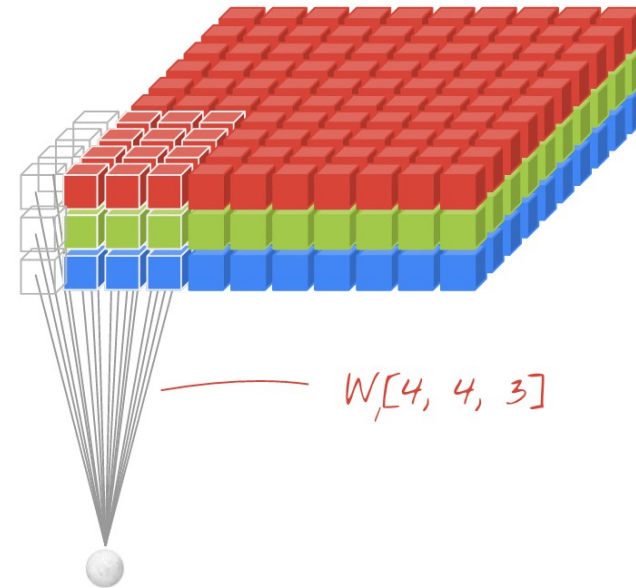
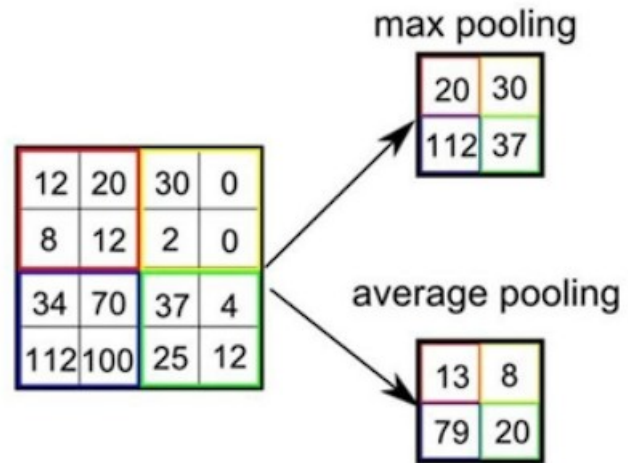
-Vorhersage, Veröffentlichung

Convolut = Konvolut/Gefaltet

Faltung



Pooling



Entstandene Probleme

- Datenqualität
- Übertrainieren/Untertrainieren
 - Verschwindende Gradienten

Verwendete Bibliotheken

- Jupyter Notebook
 - Entwicklungsumgebung
 - Direkte Visualisierung der Ausgabe oder des Ergebnisses,
- NumPy
 - Wissenschaftliche Rechnungen
 - Einfacheren Umgang mit Matrizen
 - Umsetzung einiger Funktionen der linearen Algebra.
- matplotlib
 - Bibliothek die Datenvisualisierung ermöglicht
- pandas
 - Datenverarbeitung
 - Zusammenführung
 - Zweidimensionale, SQL ähnliche, Struktur. Ermöglicht abfragen und sortieren
- scikit-learn + Tensorflow 2
 - Bibliothekensammlung (Nativ C) zur einfacheren Skalierbarkeit der Netzgröße
 - keras2onnx

Quellen und Links

[Wikipedia](#)
[Rheinwerk – Neuronale Netze mit Python](#)

[Wikimedia Commons](#)

Bilder:
[Convolute Netze](#)
[RGB Bilder](#)
[Pooling](#)
[Perzeptron Graph](#)
[Perzeptron Grundaufbau](#)